

南開大學
Nankai University

计算机学院

计算机组成原理创新实验

8 位 AI 运算器设计 week3

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 5 月 13 日

目录

1 实验目的 **2**

2 优化思路 **2**

3 优化后模块设计 **3**
3.1 固定卷积核卷积模块优化 3
3.2 平均池化模块优化 4
3.3 优化后顶层模块 4

4 功能仿真与结果分析 **5**
4.1 仿真输入与理论结果 5
4.2 优化后仿真结果 6
4.3 优化前后功能对比 7

5 综合与时序分析 **7**
5.1 资源利用率对比 7
5.2 时序结果对比 9
5.3 优化效果分析 11

6 实验总结 **11**

A Verilog 源代码 **12**
A.1 固定卷积核优化卷积模块代码 12
A.2 优化平均池化模块代码 13
A.3 优化后顶层模块代码 13
A.4 优化后系统仿真测试文件代码 16
A.5 时序约束文件 17

1 实验目的

本周实验在 Week2 完成卷积-激活-池化全链路设计的基础上，对系统中的关键运算模块进行优化，并通过 Vivado 综合和时序分析对优化效果进行验证。

Week2 中已经实现了从 5×5 输入矩阵到 2×2 输出矩阵的完整计算流程。原始设计中，3×3 卷积模块采用通用乘法累加结构，需要对 9 个输入像素和 9 个卷积核权重分别进行乘法运算，再将 9 项乘积结果累加。该结构通用性较强，但在卷积核固定的情况下会占用较多组合逻辑资源，并增加关键路径延迟。

本周实验选择对卷积模块进行固定卷积核优化。由于实验中使用的卷积核固定为：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

因此原来的 9 项乘法累加可以化简为若干加法和一次左移操作。通过这种方式，可以减少乘法相关逻辑，降低 LUT 和进位链资源占用，并缩短组合逻辑路径。

本周实验的主要目标如下：

1. 分析 Week2 原始卷积-激活-池化链式运算器中资源占用较高的部分；
2. 根据固定卷积核特点，对 3×3 卷积模块进行乘法消去优化；
3. 对平均池化模块进行简化实现，使用求和和右移完成平均池化；
4. 编写优化后的 Verilog 代码，并完成优化后系统仿真；
5. 对比优化前后仿真输出，验证优化后功能是否保持一致；
6. 使用 Vivado 对优化前后设计进行综合，比较 LUT、CARRY4、DSP 等资源占用；
7. 添加输入到输出的最大延迟约束，比较优化前后的时序结果和组合路径延迟；
8. 根据资源与时序数据，总结本次优化的效果和适用条件。

通过本周实验，可以进一步理解硬件设计中“通用性”和“资源效率”之间的关系。对于固定参数的 AI 计算场景，可以利用卷积核权重的特殊性，将复杂乘法运算转化为更简单的移位和加法运算，从而实现面向边缘端设备的低资源、低延迟优化设计。

2 优化思路

在 Week2 实验中，系统已经完成了 5×5 输入矩阵经过 3×3 卷积、ReLU 激活和 2×2 平均池化后输出 2×2 结果矩阵的完整链路。原始设计中的卷积模块采用通用乘法累加结构，即每一个 3×3 卷积窗口都需要进行 9 次乘法和多次加法累加。该方法能够支持任意卷积核，通用性较好，但对于本实验中固定卷积核的情况来说，存在一定的资源浪费。

本实验使用的卷积核为：

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

对于一个 3×3 输入窗口：

$$P = \begin{bmatrix} p00 & p01 & p02 \\ p10 & p11 & p12 \\ p20 & p21 & p22 \end{bmatrix}$$

原始卷积计算表达式为：

$$y = p00k00 + p01k01 + p02k02 + p10k10 + p11k11 + p12k12 + p20k20 + p21k21 + p22k22$$

将固定卷积核中的权重代入后，可以得到：

$$y = p01 + p10 + 2p11 + p12 + p21$$

其中，$2p11$ 可以通过左移一位实现：

$$2p11 = p11 << 1$$

因此，原来的 9 项乘法累加可以简化为 5 项加法和一次移位操作。这样可以减少乘法器相关组合逻辑，降低 LUT 和 CARRY4 进位链资源占用，同时也有利于缩短组合逻辑关键路径。

本次优化主要包含两个部分：

1. 对卷积模块进行固定卷积核优化，将原来的乘法累加结构改为加法和移位结构；
2. 对平均池化模块进行简化，直接对 4 个输入值求和后右移 2 位，完成除以 4 操作。

需要说明的是，本次卷积优化是针对当前实验固定卷积核进行的优化。优化后模块不再适用于任意卷积核输入，保留卷积核端口只是为了与原始顶层模块和测试文件接口保持一致。如果后续需要更换卷积核，则需要重新推导对应的加法、移位表达式，或者恢复通用乘法累加结构。

通过这种方式，本周实验重点比较优化前后在功能、资源占用和时序延迟方面的变化，验证固定参数条件下硬件结构优化的实际效果。

3 优化后模块设计

3.1 固定卷积核卷积模块优化

优化前的卷积模块采用通用 3×3 卷积结构，需要对 9 个像素值和 9 个卷积核权重分别相乘，再将 9 个乘积结果累加。该结构可以适用于任意卷积核，但在本实验中卷积核已经固定，因此可以根据卷积核的具体数值对运算过程进行化简。

本实验使用的固定卷积核为：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

因此，对于一个 3×3 输入窗口，卷积输出可以由原来的 9 项乘法累加简化为：

$$y = p01 + p10 + 2p11 + p12 + p21$$

其中，$2p11$ 可以通过左移一位实现，即：

$$2p11 = p11 << 1$$

优化后卷积模块不再例化乘法器，而是直接使用加法和移位完成卷积计算。这样可以减少乘法带来的组合逻辑资源占用，同时缩短卷积模块内部的运算路径。

该模块接口仍然保留 9 个输入像素信号，输出为 20 位卷积结果。虽然实际计算结果在当前测试数据下较小，但输出仍然保持 20 位，以便与 Week2 原始卷积模块的输出位宽保持一致。

需要注意的是，该优化方法只适用于当前固定卷积核。如果卷积核权重发生变化，则该模块不能直接适用，需要重新根据新的卷积核推导对应的移位加法表达式。

3.2 平均池化模块优化

Week2 中的平均池化模块通过两个 8 位加法器先分别计算两组输入之和，再将两个中间结果相加，最后调用除以 4 模块完成平均池化。该结构层次清晰，便于理解基础模块之间的连接关系。

在 Week3 中，为了进一步简化模块结构，平均池化模块直接对 4 个 8 位输入进行扩展求和，得到 10 位中间结果，然后通过右移 2 位完成除以 4 操作。其计算关系为：

$$y = (a + b + c + d) >> 2$$

由于 4 个 8 位数相加的最大值为：

$$255 + 255 + 255 + 255 = 1020$$

因此中间求和结果使用 10 位即可完整表示，不会发生溢出。优化后的池化模块减少了额外模块例化，使代码结构更加简洁。

该模块输入为 4 个 8 位数据，输出仍为 8 位平均池化结果。由于除以 4 使用右移实现，因此不需要额外的除法器资源，符合硬件设计中低资源、低延迟的优化目标。

3.3 优化后顶层模块

优化后顶层模块的整体数据流仍然保持 Week2 中的结构，即：

$$5 \times 5\text{输入矩阵} \rightarrow 3 \times 3\text{卷积结果} \rightarrow \text{ReLU 直通} \rightarrow 2 \times 2\text{平均池化输出}$$

与 Week2 不同的是，顶层模块内部不再调用通用卷积模块 `conv3x3_u8`，而是调用优化后的固定卷积核模块 `conv3x3_fixed_opt`。系统仍然需要生成 9 个卷积结果：

$$\begin{bmatrix} conv00 & conv01 & conv02 \\ conv10 & conv11 & conv12 \\ conv20 & conv21 & conv22 \end{bmatrix}$$

然后对相邻的 2×2 区域进行平均池化，得到最终输出：

$$\begin{bmatrix} y00 & y01 \\ y10 & y11 \end{bmatrix}$$

为了便于与 Week2 测试文件进行对比，优化后顶层模块仍然保留了卷积核输入端口 `k00` 到 `k22`。但是在优化后的固定卷积核实现中，这些卷积核输入信号不再参与实际计算。这样做的目的是保持模块接口形式一致，方便使用相同的输入数据进行功能对比。

通过该测试文件可以验证，优化后系统在相同输入矩阵下得到的卷积中间结果和最终池化输出与 Week2 原始设计一致，从而说明本次优化没有改变当前固定卷积核条件下的系统功能。

4 功能仿真与结果分析

4.1 仿真输入与理论结果

为了验证优化后链式运算器的功能是否正确，本实验仍然采用 Week2 中的测试输入数据。输入矩阵为一个 5×5 的 8 位无符号灰度矩阵：

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

卷积核固定为：

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

由于本周优化后的卷积模块针对该固定卷积核进行了化简，因此每一个 3×3 窗口的卷积计算可表示为：

$$y = p01 + p10 + 2p11 + p12 + p21$$

以左上角第一个卷积窗口为例：

$$\begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix}$$

其卷积结果为：

$$conv00 = 2 + 6 + 2 \times 7 + 8 + 12 = 42$$

按照相同方法，可以得到优化后理论卷积输出矩阵为：

$$C = \begin{bmatrix} 42 & 48 & 54 \\ 72 & 78 & 84 \\ 102 & 108 & 114 \end{bmatrix}$$

由于本实验仍采用无符号输入和固定非负卷积核，因此卷积结果不会出现负数，ReLU 激活层仍然等效为直通输出。随后对 3×3 卷积结果进行 2×2 平均池化，可得最终理论输出矩阵：

$$Y = \begin{bmatrix} 60 & 66 \\ 90 & 96 \end{bmatrix}$$

4.2 优化后仿真结果

在 Vivado 中运行优化后测试文件 `tb_cnn_chain_5x5_opt.v`，对优化后的顶层模块 `cnn_chain_5x5_opt` 进行行为仿真。仿真波形如图所示。

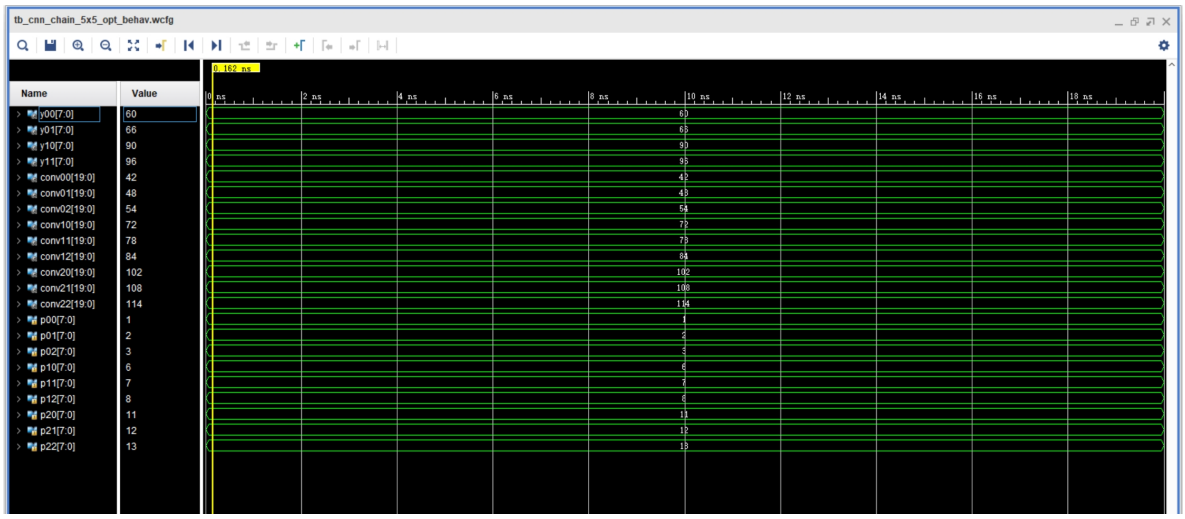


图 4.1: 优化后卷积-激活-池化链式运算器仿真波形

由仿真波形可以看出，优化后卷积模块输出结果分别为：

$$42,\ 48,\ 54,\ 72,\ 78,\ 84,\ 102,\ 108,\ 114$$

对应的 3×3 卷积输出矩阵为：

$$\begin{bmatrix} 42 & 48 & 54 \\ 72 & 78 & 84 \\ 102 & 108 & 114 \end{bmatrix}$$

最终池化输出结果分别为：

$$y00 = 60, \quad y01 = 66, \quad y10 = 90, \quad y11 = 96$$

对应输出矩阵为：

$$\begin{bmatrix} 60 & 66 \\ 90 & 96 \end{bmatrix}$$

仿真结果与理论计算结果完全一致，说明优化后的固定卷积核模块能够正确完成当前卷积核条件下的卷积计算。

4.3 优化前后功能对比

为了验证本次优化没有改变系统功能，将优化前 Week2 原始设计的输出结果与优化后设计的输出结果进行对比，如表所示。

表 1: 优化前后输出结果对比

信号名称	含义	优化前结果	优化后结果
conv00	第 1 行第 1 列卷积输出	42	42
conv01	第 1 行第 2 列卷积输出	48	48
conv02	第 1 行第 3 列卷积输出	54	54
conv10	第 2 行第 1 列卷积输出	72	72
conv11	第 2 行第 2 列卷积输出	78	78
conv12	第 2 行第 3 列卷积输出	84	84
conv20	第 3 行第 1 列卷积输出	102	102
conv21	第 3 行第 2 列卷积输出	108	108
conv22	第 3 行第 3 列卷积输出	114	114
y00	第 1 行第 1 列池化输出	60	60
y01	第 1 行第 2 列池化输出	66	66
y10	第 2 行第 1 列池化输出	90	90
y11	第 2 行第 2 列池化输出	96	96

由表可知，优化前后的卷积中间结果和最终池化输出均保持一致。这说明在当前固定卷积核条件下，将原来的乘法累加结构化简为加法和移位结构后，系统功能没有发生变化。

因此，本次优化属于功能等价优化。它没有改变系统的输入输出关系，而是在固定卷积核的前提下减少了不必要的乘法逻辑，为后续资源利用率和时序性能优化提供了基础。

5 综合与时序分析

5.1 资源利用率对比

为了比较优化前后硬件资源占用情况，本实验分别以 Week2 原始顶层模块 `cnn_chain_5x5` 和 Week3 优化后顶层模块 `cnn_chain_5x5_opt` 作为综合顶层，在 Vivado 中运行综合，并查看资源利用率报告。

优化前后设计的 Slice Logic 资源利用率对比如图所示。

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	3414	0	134600	2.54
LUT as Logic	3414	0	134600	2.54
LUT as Memory	0	0	46200	0.00
Slice Registers	0	0	269200	0.00
Register as Flip Flop	0	0	269200	0.00
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

(a) 优化前 Slice Logic 资源利用率

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	256	0	134600	0.19
LUT as Logic	256	0	134600	0.19
LUT as Memory	0	0	46200	0.00
Slice Registers	0	0	269200	0.00
Register as Flip Flop	0	0	269200	0.00
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

(b) 优化后 Slice Logic 资源利用率

图 5.2: 优化前后 Slice Logic 资源利用率对比

从资源利用率报告可以看出，优化前设计使用了 3414 个 Slice LUT，而优化后设计仅使用 256 个 Slice LUT。Slice Registers 在优化前后均为 0，这是因为当前设计为组合逻辑结构，没有引入时钟寄存器。

进一步查看综合后的 Cell Usage 统计结果，优化前和优化后的单元使用情况对比如图所示。

Report Cell Usage:

	Cell	Count
1	CARRY4	660
2	LUT2	1974
3	LUT3	63
4	LUT4	414
5	LUT6	1377
6	IBUF	272
7	OBUF	32

(a) 优化前 Cell Usage 统计

Report Cell Usage:

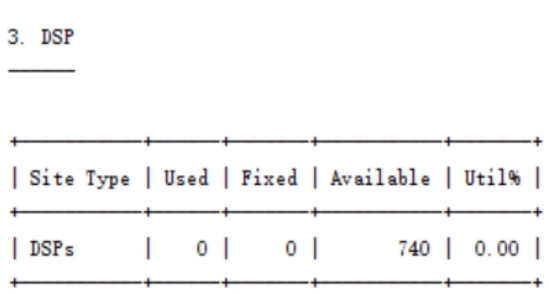
	Cell	Count
1	CARRY4	48
2	LUT2	9
3	LUT3	145
4	LUT4	102
5	LUT5	41
6	LUT6	42
7	IBUF	168
8	OBUF	32

(b) 优化后 Cell Usage 统计

图 5.3: 优化前后 Cell Usage 统计对比

优化前设计使用了 660 个 CARRY4 进位链单元，而优化后减少到 48 个。说明固定卷积核乘法消去后，不仅减少了 LUT 资源，也明显减少了加法进位链相关资源。

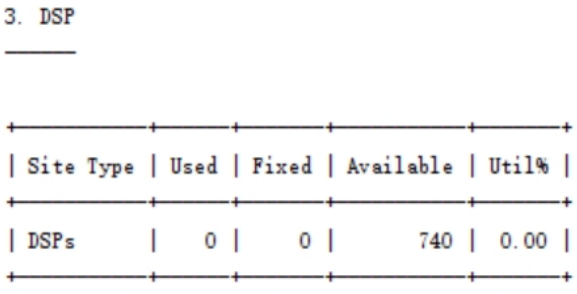
此外，优化前后 DSP 资源使用情况均为 0，如图所示。



3. DSP

Site Type	Used	Fixed	Available	Util%
DSPs	0	0	740	0.00

(a) 优化前 DSP 资源利用率



3. DSP

Site Type	Used	Fixed	Available	Util%
DSPs	0	0	740	0.00

(b) 优化后 DSP 资源利用率

图 5.4: 优化前后 DSP 资源利用率对比

这说明 Vivado 在综合原始乘法结构时，并没有将乘法映射到 DSP 资源，而是主要使用 LUT 和 CARRY4 等逻辑资源实现。因此，本次优化的主要效果体现在 LUT 和进位链资源的减少上。

优化前后主要资源对比如表所示。

表 2: 优化前后资源利用率对比

资源类型	优化前	优化后	优化效果
Slice LUTs	3414	256	减少 3158 个，约 92.50%
LUT as Logic	3414	256	减少 3158 个，约 92.50%
Slice Registers	0	0	不变
DSPs	0	0	不变
CARRY4	660	48	减少 612 个，约 92.73%

由表可知，固定卷积核优化后，Slice LUT 数量从 3414 减少到 256，减少率约为 92.50%；CARRY4 数量从 660 减少到 48，减少率约为 92.73%。这说明将原始乘法累加结构改为加法和移位结构后，能够显著降低组合逻辑资源占用。

5.2 时序结果对比

为了进一步比较优化前后的延迟情况，本实验添加了输入到输出的最大延迟约束。约束文件中加入如下语句：

```
1 set_max_delay 10.000 -from [all_inputs] -to [all_outputs]
```

该约束表示希望系统从输入到输出的最大组合路径延迟不超过 10 ns。添加约束后，分别对优化前和优化后设计重新进行综合，并查看 Timing Summary 报告。

优化前设计的时序结果如图所示。

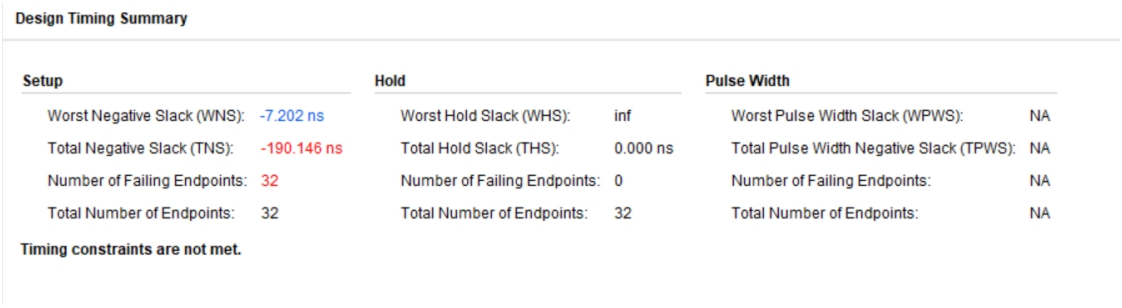


图 5.5: 优化前时序报告

由图可知，优化前设计的 WNS 为：

$$WNS = -7.202\,\text{ns}$$

在 10 ns 约束下，最大组合路径延迟可以估算为：

$$T_{before} = 10.000 - (-7.202) = 17.202\,\text{ns}$$

优化后设计的时序结果如图所示。

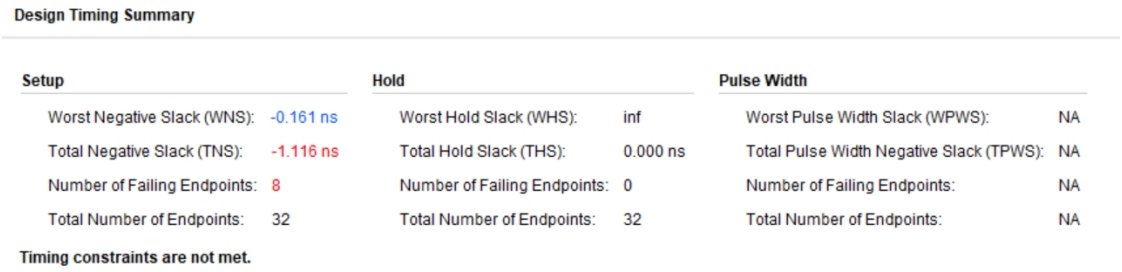


图 5.6: 优化后时序报告

由图可知，优化后设计的 WNS 为：

$$WNS = -0.161\,\text{ns}$$

因此优化后最大组合路径延迟约为：

$$T_{after} = 10.000 - (-0.161) = 10.161\,\text{ns}$$

优化前后时序结果对比如表所示。

表 3: 优化前后时序结果对比

时序指标	优化前	优化后	优化效果
WNS	-7.202 ns	-0.161 ns	改善 7.041 ns
TNS	-190.146 ns	-1.116 ns	明显改善
Failing Endpoints	32	8	减少 24 个
最大组合路径延迟	17.202 ns	10.161 ns	减少 7.041 ns，约 40.93%

可以看出，优化前设计在 10 ns 约束下存在较明显的时序违例，最大组合路径延迟约为 17.202 ns。优化后设计的最大组合路径延迟降低到约 10.161 ns，虽然仍略高于 10 ns 约束，但已经明显接近约束要求。

5.3 优化效果分析

为了更加直观地展示优化前后在资源占用和时序性能方面的综合变化，本实验进一步将主要资源指标和时序指标汇总，如表所示。

表 4: 优化前后资源与时序综合对比

类别	指标	优化前	优化后	单位	变化量/改善量	变化率	结论
资源	Slice LUTs	3414	256	个	3158	92.50%	显著减少组合逻辑资源
资源	LUT as Logic	3414	256	个	3158	92.50%	与 Slice LUTs 同步下降
资源	Slice Registers	0	0	个	0	不适用	组合逻辑设计，寄存器不变
资源	DSPs	0	0	个	0	不适用	未使用 DSP，优化主要体现在 LUT 与 CARRY4
资源	CARRY4	660	48	个	612	92.73%	明显减少进位链资源
时序	WNS	-7.202	-0.161	ns	改善 7.041 ns	–	时序裕量明显改善
时序	TNS	-190.146	-1.116	ns	改善 189.030 ns	–	总负裕量明显改善
时序	Failing Endpoints	32	8	个	24	75.00%	违例端点减少
时序	最大组合路径延迟	17.202	10.161	ns	7.041	40.93%	关键路径延迟降低
结论		优化后 Slice LUTs 从 3414 降至 256，CARRY4 从 660 降至 48，最大组合路径延迟从 17.202 ns 降至 10.161 ns，功能保持一致。					

综合资源和时序结果可以看出，本次优化效果比较明显。优化前的通用卷积模块需要进行 9 项乘法和多级加法累加，虽然功能通用，但在当前固定卷积核场景下存在一定资源浪费。优化后利用卷积核中权重只包含 0、1、2 的特点，将卷积计算化简为：

$$y = p01 + p10 + (p11 << 1) + p12 + p21$$

其中，权重为 0 的位置直接去除，权重为 1 的位置直接参与加法，权重为 2 的位置通过左移一位实现。这样避免了大量乘法逻辑，使 LUT 和 CARRY4 资源占用明显下降。

从资源角度看，Slice LUT 从 3414 减少到 256，CARRY4 从 660 减少到 48，说明优化后组合逻辑规模大幅缩小。从时序角度看，最大组合路径延迟从 17.202 ns 降低到 10.161 ns，说明优化后关键路径也得到明显缩短。

需要注意的是，本次优化是针对固定卷积核进行的。如果后续卷积核需要动态改变，则优化后的固定表达式不能直接适用，需要重新生成对应的加法移位结构，或者恢复通用乘法累加模块。因此，本次优化更适合卷积核固定、追求低资源和低延迟的边缘端 AI 计算场景。

6 实验总结

本周实验在 Week2 完整卷积-激活-池化链式运算器的基础上，对系统中的卷积模块和池化模块进行了进一步优化。Week2 中的原始卷积模块采用通用乘法累加结构，可以支持任意 3×3 卷积核，但在本实验卷积核固定的条件下，部分乘法运算实际上可以被简化。

本次优化主要利用卷积核：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

的特殊性，将原始的 9 项乘法累加化简为：

$$y = p01 + p10 + 2p11 + p12 + p21$$

其中，$2p11$ 通过左移一位实现。这样就可以去除大量乘法相关逻辑，只保留必要的加法和移位运算。平均池化模块也进一步简化为 4 个输入直接求和后右移 2 位，避免了额外除法结构。

通过功能仿真可以看出，优化后系统的卷积输出仍为：

$$\begin{bmatrix} 42 & 48 & 54 \\ 72 & 78 & 84 \\ 102 & 108 & 114 \end{bmatrix}$$

最终池化输出仍为：

$$\begin{bmatrix} 60 & 66 \\ 90 & 96 \end{bmatrix}$$

该结果与 Week2 原始设计完全一致，说明本次优化没有改变当前固定卷积核条件下的系统功能。

在资源利用率方面，优化前设计使用 3414 个 Slice LUT，优化后仅使用 256 个 Slice LUT，减少率约为 92.50%。同时，CARRY4 数量由 660 个减少到 48 个，减少率约为 92.73%。由于当前设计为组合逻辑结构，没有使用时钟寄存器，因此优化前后 Slice Registers 均为 0。综合报告中 DSP 数量也均为 0，说明 Vivado 并未将乘法映射到 DSP 资源，而是主要使用 LUT 和进位链逻辑实现。

在时序方面，本实验添加了 10 ns 输入到输出最大延迟约束。优化前最大组合路径延迟约为 17.202 ns，优化后降低到约 10.161 ns，减少了 7.041 ns，时延减少率约为 40.93%。虽然优化后仍略微超过 10 ns 约束，但相比优化前已经有明显改善，说明乘法消去优化能够有效缩短组合逻辑关键路径。

综上，本周实验完成了对 8 位 AI 链式运算器的结构优化，并通过仿真、综合和时序分析验证了优化效果。实验结果表明，在卷积核固定的边缘端 AI 计算场景中，将乘法运算转化为加法和移位运算，可以显著降低硬件资源占用，并改善组合逻辑延迟。不过，本次优化也存在一定局限性：优化后的卷积模块只适用于当前固定卷积核，如果后续需要支持不同卷积核，则需要重新推导对应的表达式，或者恢复通用乘法累加结构。

附录 A Verilog 源代码

本附录给出 Week3 优化后设计相关代码，包括固定卷积核优化卷积模块、优化平均池化模块、优化后顶层模块、系统仿真测试文件以及时序约束文件。

A.1 固定卷积核优化卷积模块代码

```
1 module conv3x3_fixed_opt (
```

```
    input [7:0] p00, p01, p02,
    input [7:0] p10, p11, p12,
    input [7:0] p20, p21, p22,
    output [19:0] y
);

wire [19:0] t01;
wire [19:0] t10;
wire [19:0] t11_shift;
wire [19:0] t12;
wire [19:0] t21;

assign t01       = {12'b0, p01};
assign t10       = {12'b0, p10};
assign t11_shift = {11'b0, p11, 1'b0};   // 等价于 p11 << 1
assign t12       = {12'b0, p12};
assign t21       = {12'b0, p21};

assign y = t01 + t10 + t11_shift + t12 + t21;

endmodule
```

A.2 优化平均池化模块代码

```
module avg_pool2x2_u8_opt (
    input [7:0] a,
    input [7:0] b,
    input [7:0] c,
    input [7:0] d,
    output [7:0] y
);

wire [9:0] sum;

assign sum = {2'b00, a} + {2'b00, b} + {2'b00, c} + {2'b00, d};
assign y = sum >> 2;

endmodule
```

A.3 优化后顶层模块代码

```
module cnn_chain_5x5_opt (
    input [7:0] p00, p01, p02, p03, p04,
    input [7:0] p10, p11, p12, p13, p14,
    input [7:0] p20, p21, p22, p23, p24,
    input [7:0] p30, p31, p32, p33, p34,
```

```
    input [7:0] p40, p41, p42, p43, p44,

    input [7:0] k00, k01, k02,
    input [7:0] k10, k11, k12,
    input [7:0] k20, k21, k22,

    output [7:0] y00,
    output [7:0] y01,
    output [7:0] y10,
    output [7:0] y11
);

wire [19:0] conv00, conv01, conv02;
wire [19:0] conv10, conv11, conv12;
wire [19:0] conv20, conv21, conv22;

// 本优化版本针对固定卷积核:
// 0 1 0
// 1 2 1
// 0 1 0
// 因此卷积计算简化为: p01 + p10 + 2*p11 + p12 + p21
// k00~k22端口保留, 是为了方便与Week2原始测试文件接口保持一致。

conv3x3_fixed_opt u_conv00 (
    .p00(p00), .p01(p01), .p02(p02),
    .p10(p10), .p11(p11), .p12(p12),
    .p20(p20), .p21(p21), .p22(p22),
    .y(conv00)
);

conv3x3_fixed_opt u_conv01 (
    .p00(p01), .p01(p02), .p02(p03),
    .p10(p11), .p11(p12), .p12(p13),
    .p20(p21), .p21(p22), .p22(p23),
    .y(conv01)
);

conv3x3_fixed_opt u_conv02 (
    .p00(p02), .p01(p03), .p02(p04),
    .p10(p12), .p11(p13), .p12(p14),
    .p20(p22), .p21(p23), .p22(p24),
    .y(conv02)
);

conv3x3_fixed_opt u_conv10 (
    .p00(p10), .p01(p11), .p02(p12),
    .p10(p20), .p11(p21), .p12(p22),
    .p20(p30), .p21(p31), .p22(p32),
    .y(conv10)
```

```
);

conv3x3_fixed_opt u_conv11 (
    .p00(p11), .p01(p12), .p02(p13),
    .p10(p21), .p11(p22), .p12(p23),
    .p20(p31), .p21(p32), .p22(p33),
    .y(conv11)
);

conv3x3_fixed_opt u_conv12 (
    .p00(p12), .p01(p13), .p02(p14),
    .p10(p22), .p11(p23), .p12(p24),
    .p20(p32), .p21(p33), .p22(p34),
    .y(conv12)
);

conv3x3_fixed_opt u_conv20 (
    .p00(p20), .p01(p21), .p02(p22),
    .p10(p30), .p11(p31), .p12(p32),
    .p20(p40), .p21(p41), .p22(p42),
    .y(conv20)
);

conv3x3_fixed_opt u_conv21 (
    .p00(p21), .p01(p22), .p02(p23),
    .p10(p31), .p11(p32), .p12(p33),
    .p20(p41), .p21(p42), .p22(p43),
    .y(conv21)
);

conv3x3_fixed_opt u_conv22 (
    .p00(p22), .p01(p23), .p02(p24),
    .p10(p32), .p11(p33), .p12(p34),
    .p20(p42), .p21(p43), .p22(p44),
    .y(conv22)
);

avg_pool2x2_u8_opt u_pool00 (
    .a(conv00[7:0]), .b(conv01[7:0]),
    .c(conv10[7:0]), .d(conv11[7:0]),
    .y(y00)
);

avg_pool2x2_u8_opt u_pool01 (
    .a(conv01[7:0]), .b(conv02[7:0]),
    .c(conv11[7:0]), .d(conv12[7:0]),
    .y(y01)
);

```

```
avg_pool2x2_u8_opt u_pool10 (
    .a(conv10[7:0]), .b(conv11[7:0]),
    .c(conv20[7:0]), .d(conv21[7:0]),
    .y(y10)
);

avg_pool2x2_u8_opt u_pool11 (
    .a(conv11[7:0]), .b(conv12[7:0]),
    .c(conv21[7:0]), .d(conv22[7:0]),
    .y(y11)
);

endmodule
```

A.4 优化后系统仿真测试文件代码

```
`timescale 1ns/1ps

module tb_cnn_chain_5x5_opt;

reg [7:0] p00, p01, p02, p03, p04;
reg [7:0] p10, p11, p12, p13, p14;
reg [7:0] p20, p21, p22, p23, p24;
reg [7:0] p30, p31, p32, p33, p34;
reg [7:0] p40, p41, p42, p43, p44;

reg [7:0] k00, k01, k02;
reg [7:0] k10, k11, k12;
reg [7:0] k20, k21, k22;

wire [7:0] y00, y01, y10, y11;

cnn_chain_5x5_opt uut (
    .p00(p00), .p01(p01), .p02(p02), .p03(p03), .p04(p04),
    .p10(p10), .p11(p11), .p12(p12), .p13(p13), .p14(p14),
    .p20(p20), .p21(p21), .p22(p22), .p23(p23), .p24(p24),
    .p30(p30), .p31(p31), .p32(p32), .p33(p33), .p34(p34),
    .p40(p40), .p41(p41), .p42(p42), .p43(p43), .p44(p44),

    .k00(k00), .k01(k01), .k02(k02),
    .k10(k10), .k11(k11), .k12(k12),
    .k20(k20), .k21(k21), .k22(k22),

    .y00(y00), .y01(y01),
    .y10(y10), .y11(y11)
);

initial begin
```

```
33      // 5x5 input matrix
34      p00 = 8'd1;  p01 = 8'd2;  p02 = 8'd3;  p03 = 8'd4;  p04 = 8'd5;
35      p10 = 8'd6;  p11 = 8'd7;  p12 = 8'd8;  p13 = 8'd9;  p14 = 8'd10;
36      p20 = 8'd11; p21 = 8'd12; p22 = 8'd13; p23 = 8'd14; p24 = 8'd15;
37      p30 = 8'd16; p31 = 8'd17; p32 = 8'd18; p33 = 8'd19; p34 = 8'd20;
38      p40 = 8'd21; p41 = 8'd22; p42 = 8'd23; p43 = 8'd24; p44 = 8'd25;
39
40      // 固定卷积核
41      k00 = 8'd0; k01 = 8'd1; k02 = 8'd0;
42      k10 = 8'd1; k11 = 8'd2; k12 = 8'd1;
43      k20 = 8'd0; k21 = 8'd1; k22 = 8'd0;
44
45      #10;
46
47      $display("Optimized convolution output:");
48      $display("%0d %0d %0d", uut.conv00, uut.conv01, uut.conv02);
49      $display("%0d %0d %0d", uut.conv10, uut.conv11, uut.conv12);
50      $display("%0d %0d %0d", uut.conv20, uut.conv21, uut.conv22);
51
52      $display("Optimized pooling output:");
53      $display("%0d %0d", y00, y01);
54      $display("%0d %0d", y10, y11);
55
56      #10;
57      $finish;
58  end
59
60  endmodule
```

A.5 时序约束文件

```
1  set_max_delay 10.000 -from [all_inputs] -to [all_outputs]
```